

Documentación del Modelo de Machine Learning para TechMind

1. Importación de Librerías y Carga de Datos

El proceso inicia con la importación de las librerías especializadas en manipulación de datos (pandas, numpy), procesamiento de lenguaje natural (**re**, **sklearn.feature_extraction.text.TfidfVectorizer**), modelos de Machine Learning (**LogisticRegression**, **MultinomialNB**), optimización (**GridSearchCV**) y evaluación de métricas y visualización (**metrics**, **matplotlib.pyplot**). A continuación, se realiza la carga del dataset **techmindv1.csv** para su análisis exploratorio inicial.

2. Preprocesamiento y Limpieza de Datos

Se implementó una función de limpieza basada en expresiones regulares que estandariza el texto a minúsculas, elimina saltos de línea, tabulaciones, URLs, dígitos y caracteres especiales no deseados. Se conservaron términos cortos de alta relevancia técnica (**como go, js, ts, ai, ml, db, etc.**) mediante un diccionario de excepciones. Posteriormente, se unificaron las columnas **titulo_limpio** y **texto_limpio** en una única columna maestra denominada **texto_completo**, la cual funge como variable de entrada (X), mientras que la columna **categoría** actúa como variable de salida o etiqueta objetivo (y).

3. División del Dataset

Los datos se dividieron utilizando un esquema de validación cruzada estratificada (**train_test_split con stratify=y**), garantizando que la proporción de las clases se mantenga constante tanto en el entrenamiento como en la prueba:

- Conjunto de Entrenamiento (**Train**): **29371** documentos (**80%**), utilizados para que el modelo aprenda los patrones de vocabulario.
- Conjunto de Prueba (**Test**): **7343** documentos (**20%**), reservados de manera independiente para evaluar la capacidad de generalización del modelo.

4. Vectorización mediante TF-IDF

Para convertir el texto plano en una matriz numérica procesable por los algoritmos, se implementó **TfidfVectorizer** con los siguientes hiperparámetros clave:

- **max_features=5000**: Limita el vocabulario a las 5,000 características más informativas.
- **stop_words='english'**: Elimina palabras vacías del idioma inglés sin valor semántico.
- **ngram_range=(1, 2)**: Captura términos individuales y pares de palabras consecutivas.
- **min_df=3**: Ignora términos con una frecuencia muy baja de aparición.

Como resultado de esta transformación, se generan las matrices **X_train_tfidf** con dimensiones de **29371** filas por **5000** columnas y **X_test_tfidf** con **7343** filas por **5000** columnas.

5. Entrenamiento de Modelos Base (Regresión Logística y Naive Bayes)

Se entrenaron dos algoritmos de clasificación de manera independiente:

- **Regresión Logística (LogisticRegression):** Configurada con **class_weight='balanced'** para contrarrestar el desequilibrio de clases, asignando un peso mayor a las categorías menos frecuentes para evitar que el modelo ignore las clases minoritarias.
- **Naive Bayes (MultinomialNB):** Implementado como modelo probabilístico alternativo de referencia.

6. Comparativa de Modelos y Selección

Tras evaluar ambos modelos sobre el conjunto de prueba de **Regresión Logística** demostró un rendimiento superior frente a **Naive Bayes** alcanzando una exactitud global de 0.7535 frente al modelo alternativo.

Al analizar el rendimiento por categoría, clases específicas como **Mobile** destacan por su alta densidad de información, obteniendo una **precisión** de **0.86** en **Regresión Logística** y **0.84** en **Naive Bayes**. La combinación de una alta precisión y un elevado *recall* en Regresión Logística garantiza una menor tasa de falsos positivos y una mayor efectividad al clasificar consultas nuevas.

7. Optimización de Hiper Parámetros (GridSearchCV)

Para maximizar el modelo seleccionado, se implementó una búsqueda en malla con validación cruzada **cv=3**, optimizador **n_jobs=-1**, evaluando combinaciones del parámetro de regularización C 0.1, 1.0, 10.0 y los solucionadores '**solver**', '**lbfgs**', '**saga**'.

El modelo optimizado arrojó métricas altamente estables:

- **Accuracy:** 0.7521
- **Precision (weighted):** 0.7522
- **Recall (weighted):** 0.7521
- **F1-Score (weighted):** 0.7516

Estos resultados confirman que el modelo optimizado mantiene un rendimiento sumamente consistente, minimizando los errores de clasificación.

8. Validación Visual mediante Matrices de Confusión

Por último, se generaron las matrices de confusión comparativas. Estas gráficas permiten corroborar visualmente que los mayores aciertos se concentran en la diagonal principal clases claramente delimitadas como *Mobile*, *Ciencia de Datos* y *Frontend*, mientras que las confusiones menores se localizan de manera lógica entre dominios técnicos estrechamente relacionados como *Backend* y *DevOps Cloud*.